DESIGN PATTERNS

1. Builder

Ne Zaman Kullanılır?

- Bir sınıfın çok fazla parametresi varsa (özellikle bazıları opsiyonelse).
- Nesnenin oluşturulma sürecinin adım adım (step-by-step) olması gerekiyorsa.
- "Constructor kirliliğini" (constructor overloading) engellemek istiyorsan.

Builder (Oluşturucu) Tasarım Deseni

Builder tasarım deseni, karmaşık nesnelerin yapısı ile oluşturulma sürecini birbirinden ayıran bir yaklaşımdır. Bu desen, aynı oluşturma süreci ile farklı gösterimlerin (farklı konfigürasyonların) elde edilmesini sağlar. Nesne oluşturma mantığını ana sınıfın dışına taşıyarak, nesnenin durumunu yöneten karmaşık "constructor" (oluşturucu) yapılarının yerini, metodolojik ve zincirleme (method chaining) bir arayüze bırakır.

Temel Avantajları:

- **Okunabilirlik:** Parametre karmaşasını engeller, kodun hangi parametrenin neyi temsil ettiğini açıkça gösterir.
- **Kontrollü İnşa:** Nesne, tüm parçaları tamamlanmadan "tamamlanmış" (final) hale geçemez; bu da nesne bütünlüğünü korur.
- **Esneklik:** Opsiyonel parametreleri yönetmek için çok sayıda kurucu metot yazma zorunluluğunu ortadan kaldırır.

```
/**
 * It makes sense to use the Builder pattern only when your products are quite
 * complex and require extensive configuration.
 *
 * Unlike in other creational patterns, different concrete builders can produce
 * unrelated products. In other words, results of various builders may not
 * always follow the same interface.
 */

class Product1{
    public:
    std::vector<std::string> parts_;
    void ListParts()const{
        std::cout << "Product parts: ";
        for (size_t i=0;i<parts_.size();i++){
            if(parts_[i]== parts_.back()){
                std::cout << parts_[i];
            }else{
                std::cout << parts_[i] << ", ";
            }
        }
        std::cout << "\n\n";
    }
};


/**
 * The Builder interface specifies methods for creating the different parts of
 * the Product objects.
 */
class Builder{
    public:
    virtual ~Builder(){}
```

```
    virtual void ProducePartA() const =0;
    virtual void ProducePartB() const =0;
    virtual void ProducePartC() const =0;
};
/**
 * The Concrete Builder classes follow the Builder interface and provide
 * specific implementations of the building steps. Your program may have several
 * variations of Builders, implemented differently.
 */
class ConcreteBuilder1 : public Builder{
    private:

    Product1* product;

    /**
     * A fresh builder instance should contain a blank product object, which is
     * used in further assembly.
     */
    public:

    ConcreteBuilder1(){
        this->Reset();
    }

    ~ConcreteBuilder1(){
        delete product;
    }

    void Reset(){
        this->product= new Product1();
    }
    /**
     * All production steps work with the same product instance.
     */

    void ProducePartA()const override{
        this->product->parts_.push_back("PartA1");
    }

    void ProducePartB()const override{
        this->product->parts_.push_back("PartB1");
    }

    void ProducePartC()const override{
        this->product->parts_.push_back("PartC1");
    }

    /**
     * Concrete Builders are supposed to provide their own methods for
     * retrieving results. That's because various types of builders may create
     * entirely different products that don't follow the same interface.
     * Therefore, such methods cannot be declared in the base Builder interface
     * (at least in a statically typed programming language). Note that PHP is a
     * dynamically typed language and this method CAN be in the base interface.
     * However, we won't declare it there for the sake of clarity.
     *
     * Usually, after returning the end result to the client, a builder instance
     * is expected to be ready to start producing another product. That's why
     * it's a usual practice to call the reset method at the end of the
     * `getProduct` method body. However, this behavior is not mandatory, and
     * you can make your builders wait for an explicit reset call from the
     * client code before disposing of the previous result.
     */

    /**
     * Please be careful here with the memory ownership. Once you call
     * GetProduct the user of this function is responsable to release this
     * memory. Here could be a better option to use smart pointers to avoid
     * memory leaks
     */

    Product1* GetProduct() {
        Product1* result= this->product;
        this->Reset();
        return result;
    }
```

```
};
/**
 * The Director is only responsible for executing the building steps in a
 * particular sequence. It is helpful when producing products according to a
 * specific order or configuration. Strictly speaking, the Director class is
 * optional, since the client can control builders directly.
 */
class Director{
    /**
     * @var Builder
     */
    private:
    Builder* builder;
    /**
     * The Director works with any builder instance that the client code passes
     * to it. This way, the client code may alter the final type of the newly
     * assembled product.
     */

    public:

    void set_builder(Builder* builder){
        this->builder=builder;
    }

    /**
     * The Director can construct several product variations using the same
     * building steps.
     */

    void BuildMinimalViableProduct(){
        this->builder->ProducePartA();
}
    void BuildFullFeaturedProduct(){
        this->builder->ProducePartA();
        this->builder->ProducePartB();
        this->builder->ProducePartC();
    }
};
/**
 * The client code creates a builder object, passes it to the director and then
 * initiates the construction process. The end result is retrieved from the
 * builder object.
 */
/**
 * I used raw pointers for simplicity however you may prefer to use smart
 * pointers here
 */
void ClientCode(Director& director)
{
    ConcreteBuilder1* builder = new ConcreteBuilder1();
    director.set_builder(builder);
    std::cout << "Standard basic product:\n";
    director.BuildMinimalViableProduct();

    Product1* p= builder->GetProduct();
    p->ListParts();
    delete p;

    std::cout << "Standard full featured product:\n";
    director.BuildFullFeaturedProduct();

    p= builder->GetProduct();
    p->ListParts();
    delete p;

    // Remember, the Builder pattern can be used without a Director class.
    std::cout << "Custom product:\n";
    builder->ProducePartA();
    builder->ProducePartC();
    p=builder->GetProduct();
    p->ListParts();
    delete p;

    delete builder;
}
```

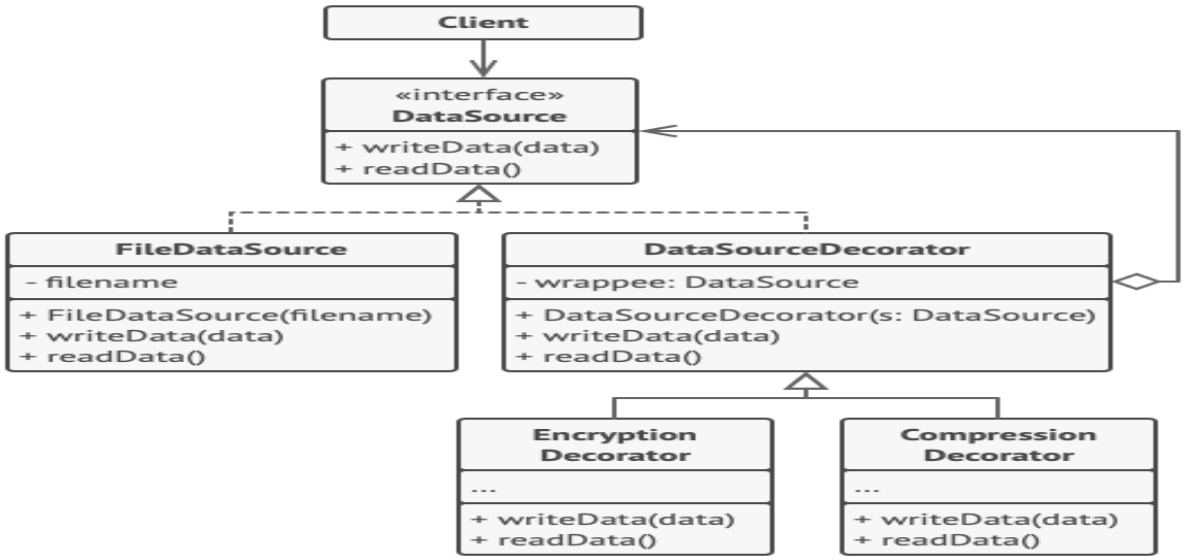
```
int main(){
    Director* director= new Director();
    ClientCode(*director);
    delete director;
    return 0;
}
Output:
Standard basic product:
Product parts: PartA1

Standard full featured product:
Product parts: PartA1, PartB1, PartC1

Custom product:
Product parts: PartA1, PartC1
```

2. Decorator

Dekoratör tasarım deseni, bir nesnenin temel işlevselliğini bozmadan veya değiştirmeden, ona çalışma zamanında dinamik olarak ek işlevler kazandırmayı amaçlayan bir **yapısal (structural)** tasarım desenidir. Kalıtım yoluyla oluşturulabilecek sınıfların sayısındaki üstel artışı (sınıf patlamasını) önler ve nesneye esnek bir şekilde özelliklerin eklenip çıkarılmasını sağlar. **Runtime** çalışır.

Temel Özellikleri:

- **Dinamik Genişletilebilirlik:** Nesneler oluşturulduktan sonra bile yeni davranışlar eklenebilir.
- **Sorumluluk Ayrımı:** Temel görevler ile dekoratörlerin eklediği ek özellikler birbirinden ayrı tutulur (Single Responsibility).
- **Şeffaflık:** Dekoratör, sarıp sarmaladığı nesne ile aynı arayüzü (interface) uygular; bu nedenle sistem, dekoratörün varlığını fark etmeden nesneyi orijinaliymiş gibi kullanmaya devam eder.




```
/**
 * The base Component interface defines operations that can be altered by
 * decorators.
```

```
 */
class Component {
 public:
  virtual ~Component() {}
  virtual std::string Operation() const = 0;
};
/**
 * Concrete Components provide default implementations of the operations. There
 * might be several variations of these classes.
 */
class ConcreteComponent : public Component {
 public:
  std::string Operation() const override {
    return "ConcreteComponent";
  }
};
/**
 * The base Decorator class follows the same interface as the other components.
 * The primary purpose of this class is to define the wrapping interface for all
 * concrete decorators. The default implementation of the wrapping code might
 * include a field for storing a wrapped component and the means to initialize
 * it.
 */
class Decorator : public Component {

protected:
  Component* component_;
 public:
  Decorator(Component* component) : component_(component) {
  }
  /**
   * The Decorator delegates all work to the wrapped component.
   */
  std::string Operation() const override {
    return this->component_->Operation();
  }
};
 * Concrete Decorators call the wrapped object and alter its result in some way.

class ConcreteDecoratorA : public Decorator {
   * Decorators may call parent implementation of the operation, instead of
   * calling the wrapped object directly. This approach simplifies extension of
   * decorator classes.

public:
  ConcreteDecoratorA(Component* component) : Decorator(component) {
  }
  std::string Operation() const override {
    return "ConcreteDecoratorA(" + Decorator::Operation() + ")";
  }
};
* Decorators can execute their behavior either before or after the call to
 * a wrapped object.
class ConcreteDecoratorB : public Decorator {
 public:
  ConcreteDecoratorB(Component* component) : Decorator(component) {
  }
  std::string Operation() const override {
    return "ConcreteDecoratorB(" + Decorator::Operation() + ")";
  }
```

```
};
* The client code works with all objects using the Component interface. This
 * way it can stay independent of the concrete classes of components it works
 * with.
void ClientCode(Component* component) {
  // ...
  std::cout << "RESULT: " << component->Operation();
  // ...
}

int main() {
  /**
   * This way the client code can support both simple components...
   */
  Component* simple = new ConcreteComponent;
  std::cout << "Client: I've got a simple component:\n";
  ClientCode(simple);
  std::cout << "\n\n";
   * ...as well as decorated ones.
   *
   * Note how decorators can wrap not only simple components but the other
   * decorators as well.
  Component* decorator1 = new ConcreteDecoratorA(simple);
  Component* decorator2 = new ConcreteDecoratorB(decorator1);

//bu satırlarda nasıl decorate edildiğini takip edebilirsin!

  std::cout << "Client: Now I've got a decorated component:\n";
  ClientCode(decorator2);
  std::cout << "\n";

  delete simple;
  delete decorator1;
  delete decorator2;
  return 0;
}
```

Output.txt: Execution result

```
Client: I've got a simple component:
RESULT: ConcreteComponent

Client: Now I've got a decorated component:
RESULT: ConcreteDecoratorB(ConcreteDecoratorA(ConcreteComponent))
```

3. Composite

Kompozit tasarım deseni, nesneleri ağaç (tree) yapılarına dönüştürerek "parça-bütün" (part-whole) hiyerarşilerini temsil etmeyi sağlayan **yapısal (structural)** bir tasarım desenidir.

Amacı ve İşlevi: İstemcilerin (client sınıfların), tekil nesneler (yaprak/leaf) ile bu nesnelerin birleşiminden oluşan karmaşık yapıları (kompozit/composite) tek tipte, aynı arayüz üzerinden işlemesini sağlar. İstemci, o an etkileşime girdiği nesnenin basit bir eleman mı yoksa alt elemanlara sahip karmaşık bir düğüm mü olduğunu bilmek zorunda kalmaz. Bu durum,

özellikle hiyerarşik veya özyineli (recursive) veri yapılarıyla çalışırken kod tekrarını önler ve if-else bloklarını ortadan kaldırarak polimorfizmi maksimize eder.

```
class Graphic {
public:
    virtual void move(int x, int y) = 0;
    virtual void draw() = 0;
    virtual ~Graphic() {}
};

class Dot : public Graphic {
protected:
    int x, y;
public:
    Dot(int x, int y) {
        this->x = x;
        this->y = y;
    }

    void move(int x, int y) override {

        this->x += x;
        this->y += y;
    }

    void draw() override {
        cout << "Nokta cizildi: " << x << ", " << y << endl;
    }
};

class Circle : public Dot {
private:
    int radius;
public:

    Circle(int x, int y, int radius) : Dot(x, y) {
        this->radius = radius;
    }

    void draw() override {
        cout << "Daire cizildi: " << x << ", " << y << " yaricap: " << radius <<
endl;
    }
};
```

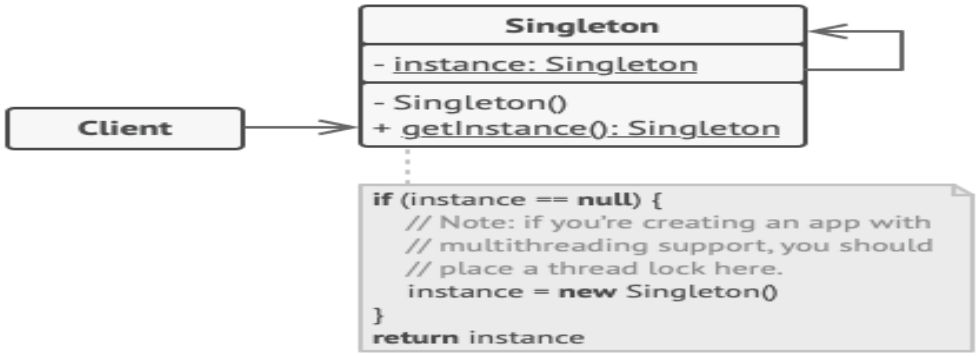
```
class CompoundGraphic : public Graphic {
private:
    vector<Graphic*> children;

public:
    void add(Graphic* child) {

        children.push_back(child);
    }

    void remove(Graphic* child) {
        auto it = find(children.begin(), children.end(), child);
        if(it != children.end()) {
            children.erase(it);
        }
    }

    void move(int x, int y) override {
        for(Graphic* child : children) {

            child->move(x, y);
        }
    }

    void draw() override {
        for(Graphic* child : children) {
            child->draw();
        }

        cout << "Kesikli cizgilerle cevre cizildi" << endl;
    }
};
```

```
class ImageEditor {
public:
    CompoundGraphic* all;

    ImageEditor() {
        all = new CompoundGraphic();
    }

    void load() {
        all->add(new Dot(1, 2));
        all->add(new Circle(5, 3, 10));
    }

    void groupSelected(vector<Graphic*> components) {

        CompoundGraphic* group = new CompoundGraphic();

        for(Graphic* component : components) {
            group->add(component);
            all->remove(component);
        }

        all->add(group);

        all->draw();
    }
};
```

4. Singleton

- Class'ın tek instance/objesi olduğuna emin ol.
- Bu objeye/instanceye bir global pointer ile işaret et.

➔ Bu class'ın default constructor'ını private//özel yap ki başka objelerin "new" operatorunu kullanması mümkün olmasın.




```
/**
 * The Singleton class defines the `GetInstance` method that serves as an
 * alternative to constructor and lets clients access the same instance of this
 * class over and over.
 */
class Singleton
{

    /**
     * The Singleton's constructor should always be private to prevent direct
     * construction calls with the `new` operator.
     */

protected:
    Singleton(const std::string value): value_(value)
    {
    }

    static Singleton* singleton_;

    std::string value_;

public:
    /**
     * Singletons should not be cloneable.
     */
    Singleton(Singleton &other) = delete;
    /**
     * Singletons should not be assignable.
     */
    void operator=(const Singleton &) = delete;
    /**
     * This is the static method that controls the access to the singleton
     * instance. On the first run, it creates a singleton object and places it
     * into the static field. On subsequent runs, it returns the client existing
     * object stored in the static field.
     */

    static Singleton *GetInstance(const std::string& value);
    /**
```

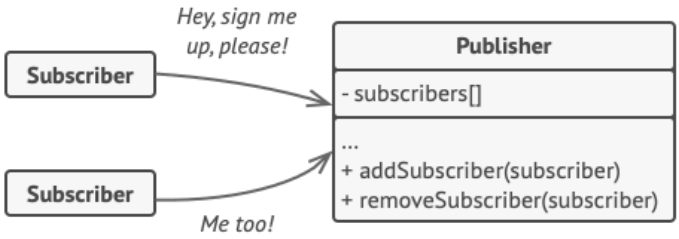
```
    * Finally, any singleton should define some business logic, which can be
    * executed on its instance.
    */
    void SomeBusinessLogic()
    {
        // ...
    }
    std::string value() const{
        return value_;
    }
};

Singleton* Singleton::singleton_= nullptr;;

/**
 * Static methods should be defined outside the class.
 */
Singleton *Singleton::GetInstance(const std::string& value)
{
    /**
     * This is a safer way to create an instance. instance = new Singleton is
     * dangeruous in case two instance threads wants to access at the same time
     */
    if(singleton_==nullptr){
        singleton_ = new Singleton(value);
    }
    return singleton_;
}
void ThreadFoo(){
    // Following code emulates slow initialization.
    std::this_thread::sleep_for(std::chrono::milliseconds(1000));
    Singleton* singleton = Singleton::GetInstance("FOO");
    std::cout << singleton->value() << "\n";
}
void ThreadBar(){
    // Following code emulates slow initialization.
    std::this_thread::sleep_for(std::chrono::milliseconds(1000));
    Singleton* singleton = Singleton::GetInstance("BAR");
    std::cout << singleton->value() << "\n";
}


int main()
{
    std::cout <<"If you see the same value, then singleton was reused (yay!\n" <<
              "If you see different values, then 2 singletons were created (booo!!)\n\n" <<
              "RESULT:\n";
    std::thread t1(ThreadFoo);
    std::thread t2(ThreadBar);
    t1.join();
    t2.join();

    return 0;
}
```

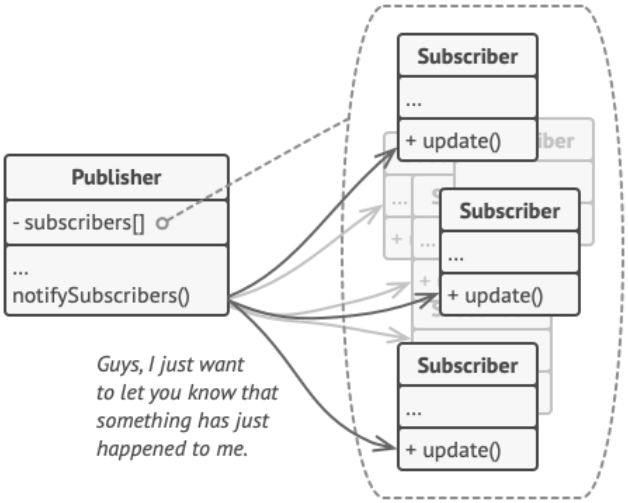
Output.txt: Execution result

```
If you see the same value, then singleton was reused (yay!
If you see different values, then 2 singletons were created (booo!!)

RESULT:
BAR
FOO
```

5. Observer

- Subscriber ve Publisher denen iki üyeden oluşur. *Subscriberler* Publisher'de herhangi bir değişim olduğunda bildirim alan, bu değişimin takipçisi olan kullanıcılar/üyelerdir. *Publisher* ise durumu/state'I değiştiğinde üyelerini habere den kişidir.






```
/**
 * Observer Design Pattern
 *
 * Intent: Lets you define a subscription mechanism to notify multiple objects
 * about any events that happen to the object they're observing.
 *
 * Note that there's a lot of different terms with similar meaning associated
 * with this pattern. Just remember that the Subject is also called the
 * Publisher and the Observer is often called the Subscriber and vice versa.
 * Also the verbs "observe", "listen" or "track" usually mean the same thing.
 */

#include <iostream>
#include <list>
#include <string>

class IObserver {
 public:
  virtual ~IObserver(){};
  virtual void Update(const std::string &message_from_subject) = 0;
};

class ISubject {
 public:
```

```
  virtual ~ISubject(){};
  virtual void Attach(IObserver *observer) = 0;
  virtual void Detach(IObserver *observer) = 0;
  virtual void Notify() = 0;
};

/**
 * The Subject owns some important state and notifies observers when the state
 * changes.
 */

class Subject : public ISubject {
 public:
  virtual ~Subject() {
    std::cout << "Goodbye, I was the Subject.\n";
  }

  /**
   * The subscription management methods.
   */
  void Attach(IObserver *observer) override {
    list_observer_.push_back(observer);
  }
  void Detach(IObserver *observer) override {
    list_observer_.remove(observer);
  }
  void Notify() override {
    std::list<IObserver *>::iterator iterator = list_observer_.begin();
    HowManyObserver();
    while (iterator != list_observer_.end()) {
      (*iterator)->Update(message_);
      ++iterator;
    }
  }

  void CreateMessage(std::string message = "Empty") {
    this->message_ = message;
    Notify();
  }
  void HowManyObserver() {
    std::cout << "There are " << list_observer_.size() << " observers in the list.\n";
  }

  /**
   * Usually, the subscription logic is only a fraction of what a Subject can
   * really do. Subjects commonly hold some important business logic, that
   * triggers a notification method whenever something important is about to
   * happen (or after it).
   */
  void SomeBusinessLogic() {
    this->message_ = "change message message";
    Notify();
    std::cout << "I'm about to do some thing important\n";
  }

 private:
  std::list<IObserver *> list_observer_;
  std::string message_;
};
```

```
class Observer : public IObserver {
 public:
  Observer(Subject &subject) : subject_(subject) {
    this->subject_.Attach(this);
    std::cout << "Hi, I'm the Observer \"" << ++Observer::static_number_ << "\".\n";
    this->number_ = Observer::static_number_;
  }
  virtual ~Observer() {
    std::cout << "Goodbye, I was the Observer \"" << this->number_ << "\".\n";
  }

  void Update(const std::string &message_from_subject) override {
    message_from_subject_ = message_from_subject;
    PrintInfo();
  }
  void RemoveMeFromTheList() {
    subject_.Detach(this);
    std::cout << "Observer \"" << number_ << "\" removed from the list.\n";
  }
  void PrintInfo() {
    std::cout << "Observer \"" << this->number_ << "\": a new message is available --> " << this-
>message_from_subject_ << "\n";
  }

 private:
  std::string message_from_subject_;
  Subject &subject_;
  static int static_number_;
  int number_;
};

int Observer::static_number_ = 0;

void ClientCode() {
  Subject *subject = new Subject;
  Observer *observer1 = new Observer(*subject);
  Observer *observer2 = new Observer(*subject);
  Observer *observer3 = new Observer(*subject);
  Observer *observer4;
  Observer *observer5;

  subject->CreateMessage("Hello World! :D");
  observer3->RemoveMeFromTheList();

  subject->CreateMessage("The weather is hot today! :p");
  observer4 = new Observer(*subject);

  observer2->RemoveMeFromTheList();
  observer5 = new Observer(*subject);

  subject->CreateMessage("My new car is great! ;)");
  observer5->RemoveMeFromTheList();

  observer4->RemoveMeFromTheList();
  observer1->RemoveMeFromTheList();

  delete observer5;
  delete observer4;
  delete observer3;
  delete observer2;
```

```
  delete observer1;
  delete subject;
}

int main() {
  ClientCode();
  return 0;
}
```

Output.txt: Execution result

```
Hi, I'm the Observer "1".
Hi, I'm the Observer "2".
Hi, I'm the Observer "3".
There are 3 observers in the list.
Observer "1": a new message is available --> Hello World! :D
Observer "2": a new message is available --> Hello World! :D
Observer "3": a new message is available --> Hello World! :D
Observer "3" removed from the list.
There are 2 observers in the list.
Observer "1": a new message is available --> The weather is hot today! :p
Observer "2": a new message is available --> The weather is hot today! :p
Hi, I'm the Observer "4".
Observer "2" removed from the list.
Hi, I'm the Observer "5".
There are 3 observers in the list.
Observer "1": a new message is available --> My new car is great! ;)
Observer "4": a new message is available --> My new car is great! ;)
Observer "5": a new message is available --> My new car is great! ;)
Observer "5" removed from the list.
Observer "4" removed from the list.
Observer "1" removed from the list.
Goodbye, I was the Observer "5".
Goodbye, I was the Observer "4".
Goodbye, I was the Observer "3".
Goodbye, I was the Observer "2".
Goodbye, I was the Observer "1".
Goodbye, I was the Subject.
```